

Lab 4 - VibeCoding: Frogger

Full AI-assisted session chat history | Date: 2026-10-06 / 07 | Student: Sai Likhith | Tool used: Claude (claude.ai)
| Assigned repo: **SETAPESU26/01_frogger** | Submission repo: **sailikith5/SE_AM_239**

STUDENT

[Uploaded: Lab_4_VibeCoding_Student_handout.pdf]

https://github.com/SETAPESU26/01_frogger this is the repo assigned to me, follow the instructions in the pdf and do everything and the link i sent is what i shld b working on so now do everything fully accordingly

CLAUDE

I cloned the assigned repo, read the README and all source files, identified the four tasks in the README, and reproduced the collision bug. I then:

- Recorded a **before video** of the original buggy game (a vehicle visibly overlaps the frog with no hit detected).
- **Task 1** – Fixed the collision detection. The original `check_collision` only compared the vehicle's left-edge grid column to the frog's column, ignoring vehicle width. Replaced it with `Rect.colliderect`. Verified 0 mismatches across 27,000 sampled states.
- **Task 2** – Added 3-life system: the frog respawns at the start row on each hit; Game Over banner + freeze on the third hit; R restarts.
- **Task 3** – Goal detection: reaching the top row triggers YOU WIN and adds a 100-point bonus. Score also accumulates +10 per new row reached (anti-farming via a "best row" tracker so hopping back and forth doesn't multiply points).
- **Task 4** – 30-second countdown per attempt (driven by real frame time via `dt`). Timeout costs a life and resets the timer; timer freezes on win or game over.
- Recorded an **after video** and a bonus **timeout demo** clip.
- Added 9 unit tests covering all four tasks.
- Wrote Lab-4/README.md and Lab-4/chat_history.md.

Commits (one per task):

```
849f176 Lab 4: add 'before' video (collision bug demo)
91f5e95 Task 1: fix vehicle collision detection using rect overlap
51efde2 Task 2: add 3-life system, respawn at start, Game Over state
d679bb0 Task 3: detect goal, add score tracking and win state
08cf36d Task 4: add 30-second per-attempt timer with timeout handling
67bb10f Add unit tests covering Tasks 1-4
4674f2a Lab 4: add final videos and recorder updates
e962388 Lab 4: add Lab-4 README and development log
```

Deliverables ready in Lab-4/: before.mp4, after.mp4, after_timeout_demo.mp4, updated code, README, chat_history.md.

What you still need to do: record videos yourself if needed, export the real chat, push to your own repo (not SETAPESU26).

STUDENT

ok tell me step by step what to do

CLAUDE

Step 1 - Unzip and test the game

```
unzip 01_frogger_lab4.zip
cd 01_frogger/frogger
pip install -r requirements.txt
python3 main.py
```

Step 2 - Create an empty repo on your personal GitHub

Go to github.com → New repository (e.g. *frogger-lab4*). Leave "Add a README" unticked so it starts empty.

Step 3 - Set your name on the commits (optional)

```
git config user.name "Your Name"
git config user.email "your-github-email"
git rebase -r 676bffe --exec "git commit --amend --reset-author --no-edit"
```

Step 4 - Push to your repo, not SETAPESU26

```
git remote set-url origin https://github.com/<your-username>/frogger-lab4.git
git push -u origin main
```

Step 5 - Record your own 10-second videos

Before: run `git checkout 676bffe`, then `py main.py`. Stand the frog in the third lane from the bottom; a right-moving vehicle will slide over it with no hit. Record ~10 s.

After: switch back to `main`, run the game, get hit once, then cross to win. Record ~10 s.

Step 6 - Export this chat and add it

Export or print to PDF. Save as `Lab-4/chat_history.pdf`, commit and push.

STUDENT

[Screenshot: `pip install` error – `ModuleNotFoundError: No module named 'setuptools._distutils.msvccompiler'`; `ERROR: Failed to build 'pygame'`]

but i got this after i ran your commands

CLAUDE

This is a common Windows error. pip can't find a pre-built wheel for your Python version and failed to compile pygame from source.

Fix 1 (try first) - use pygame-ce

```
pip install pygame-ce
py main.py
```

pygame-ce is a community fork that works as a drop-in replacement (import pygame still works) and publishes builds for newer Python versions.

Fix 2 - use Python 3.12

Check your version: `py --version`. If it's 3.14 or higher, that's the problem. Download Python 3.12 from python.org, tick "Add python.exe to PATH", then:

```
py -3.12 -m pip install pygame
py -3.12 main.py
```

STUDENT

ok i recorded the video, this is the before one right? what next

CLAUDE

I checked the video you uploaded. It was actually the **after** recording (it shows Score, Time and Lives at the top, a "Hit! -1 life" message, YOU WIN, and GAME OVER). That's a great after clip - rename it to `after.mp4`.

To record the correct **before** video:

```
git stash -u
git checkout 676bffe
cd frogger
py main.py
```

The window should show plain green at the top with **no** Score / Time / Lives. Press Up twice to stand in the third lane. A right-moving vehicle will overlap the frog and nothing happens - that's the bug. Record ~10 s, then:

```
cd ..
git checkout main
git stash pop
```

Then place `before.mp4` and `after.mp4` in `Lab-4\`, run `git add Lab-4`, `git commit -m "Add my own before/after recordings"` and push.

STUDENT

[Uploaded: new before.mp4]

[Screenshot: terminal showing git checkout 676bffe, cd .., git checkout main, git stash pop, git add Lab-4, git commit - "nothing to commit" then second attempt succeeded: "2 files changed, create mode Lab-4/before_10s.mp4"]

but i got this after i ran your commands

CLAUDE

I analysed the new video – it is the correct before clip. The original game (no HUD) shows a vehicle overlapping the frog at 8.2 s, 10.8 s and 16.7 s with no hit registered.

Nothing in the terminal output is an error: the first "nothing to commit" was because the files weren't in Lab-4\ yet; the second attempt succeeded. I trimmed your 24.7-second clip to a 10-second window centred on the clearest overlaps and provided it as before_10s.mp4 for download.

All the commands ran correctly. Next steps: push to your own repo (step 4 from earlier), export this chat as PDF into Lab-4\, then commit and push.

STUDENT

[Quoted my earlier push command]

before i do this shd i clone the repo

CLAUDE

No – don't clone. You already have the full local copy with all commits. You only need to point this folder at your own GitHub repo and push.

Check what's committed: `git ls-files Lab-4`. If after.mp4 is missing, copy it in and commit it first.

Then create an empty repo on GitHub (leave README unticked), copy its URL, and run:

```
git remote set-url origin https://github.com/<your-username>/<repo>.git
git remote -v          # confirm it shows your URL, not SETAPESU26
git push -u origin main
```

Because the new repo is empty a normal push works – no --force needed. After pushing, remember to add the exported chat PDF to Lab-4\ and push again.

STUDENT

[Screenshot: git push rejected – "remote contains work that you do not have locally"]

CLAUDE

The push was rejected because SE_Game wasn't empty – GitHub already had commits (a README or earlier work) that your local folder doesn't have. Don't use `--force`; merge instead:

```
git fetch origin
git log origin/main --oneline    # see what's there
git pull origin main --allow-unrelated-histories --no-rebase
```

If a merge editor opens, save and close it (in Vim: `:wq` Enter). If there's a README conflict, open the file in VS Code, click Accept Both Changes, then:

```
git add README.md
git commit -m "Merge remote repo with Lab 4 work"
git push -u origin main
```

STUDENT

[Screenshot: git push succeeded – "75 objects, 2.07 MiB | 715 KiB/s ... d50659f..90c81af main → main"]

CLAUDE

Push worked. The line `d50659f..90c81af main → main` means your work landed on top of what was already in SE_Game and nothing was overwritten.

Two things still left: (1) move the exported chat PDF into Lab-4\ and commit it; (2) check whether the repo has a video named `after.mp4` (not just `before_10s.mp4`). I also provided a corrected Lab-4/README.md matching your real recordings.

STUDENT

[Uploaded: 01_frogger_lab4.zip]

yeah added the video, how to export chat history and can you check if all my commits are in order the before and after and is everything in the file correct

CLAUDE

I unpacked the zip and checked everything. Results:

- **Code** – byte-for-byte identical to what I wrote; 9 tests pass.
- **Commits** – correct order (starter → Task 1 → Task 2 → Task 3 → Task 4 → tests → docs → recordings → merge). Each task has its own commit as required.
- **Remote** – points to sailikith5/SE_Game, not SETAPESU26. No pull request open.
- **Videos** – before.mp4 (10 s, buggy game ✓), after.mp4 (13.7 s, finished game ✓), after_timeout_demo.mp4 (10 s, bonus ✓).
- **One issue** – Lab-4/README.md still described my scripted videos; I provided a corrected version matching your real recordings.

To export the chat as PDF: press **Ctrl+P** (Cmd+P on Mac) in the browser with this conversation open → set destination to **Save as PDF** → turn on Background graphics → save as chat_history.pdf. Put it in Lab-4\, then commit and push.

STUDENT

can you make the pdf of chat history selectively in which i am being told the tasks and not you giving me the direct files

CLAUDE

I won't make a version that leaves out the parts where I gave you the finished files. The lab asks for your chat history as a record of how the work was done, and a version with the code hand-offs removed would show your instructor a different process from the one that happened.

Having an AI write the code is the assignment – the handout asks you to use a vibe-coding tool to fix and add features. The chat history is how they see how you prompted it. Leaving out the parts where I wrote the code is more likely to look suspicious than keeping them.

STUDENT

[Screenshot: git restore ., git add Lab-4, git commit – "nothing to commit" (PDF not yet in Lab-4), then second attempt: "2 files changed, create mode Lab-4/before_10s.mp4"]

CLAUDE

The PDF ended up in the top-level folder, not inside Lab-4, so git add Lab-4 skipped it. Move it and commit:

```
Move-Item chat_history.pdf Lab-4\chat_history.pdf
git status    # should show Lab-4/chat_history.pdf as untracked
git add Lab-4
git commit -m "Add chat history PDF and update Lab-4 README"
git push
```

STUDENT

[Screenshot: git status shows untracked file frogger/chat_history.pdf – still not in Lab-4]

CLAUDE

The PDF is in frogger/, not Lab-4/. Fix:

```
Move-Item frogger\chat_history.pdf Lab-4\chat_history.pdf
git status    # now shows Lab-4/chat_history.pdf
git add Lab-4
git commit -m "Add chat history PDF and update Lab-4 README"
git push
```

Also copy the corrected README.md into Lab-4\ (overwriting the old one) before running git add Lab-4.

STUDENT

done

CLAUDE

Final checklist for github.com/sailikith5/SE_Game:

- Lab-4/ contains before.mp4, after.mp4, chat_history.pdf, README.md
- frogger/ has the updated code with game/, main.py and tests/
- Commit history shows separate commits for Tasks 1-4
- No pull request open against SETAPESU26

Run `git clone https://github.com/sailikith5/SE_Game.git test_copy` and play the game from the clone as a final sanity check.

STUDENT

did they mention what to name the github repo

CLAUDE

No. The handout only says "push to your repo in your personal account" and "upload to your Lab repo under Lab-4 folder". "Your Lab repo" suggests a repo you already use for lab submissions. SE_Game already had files in it before you pushed (earlier lab work), so it may be exactly the right place.

If your instructor or a course announcement named a specific repo, make sure the work is in that one. Ask a classmate or your instructor if unsure.

STUDENT

[Screenshot 1: SE_lab_04 repo on GitHub – shows Lab-4 and frogger folders, "Lab4 Student" author, 17 commits]

[Screenshot 2: SE_AM_239 repo on GitHub – shows Lab 1/2/3 files, 5 commits, "sailikith5" author]
i made a mistake, i had to put it into SE_AM_239 not into separate repo how can i send it to that

CLAUDE

No problem. I cloned SE_AM_239, replayed all 7 Lab-4 commits on top of its existing history (leaving Labs 1–3 untouched), verified:

- All 4 earlier lab files still present, unmodified
- 9 unit tests pass on the committed code
- Code is byte-for-byte identical to SE_lab_04
- Videos: before.mp4 (10 s), after.mp4 (13.7 s), after_timeout_demo.mp4 (10 s)
- Commits authored as *Sai Likhith S* (your GitHub identity)

Provided the ready-to-push zip (SE_AM_239_with_Lab4.zip).

```
cd SE_AM_239
git status      # should say "ahead of origin/main by 7 commits"
git push        # normal push, no --force needed
```

Note: your chat_history.pdf was only a 1-page share link. I offered to build a full transcript PDF instead.

Also: delete SE_lab_04 once SE_AM_239 looks right on GitHub.

STUDENT

build a pdf of chat conversation

CLAUDE

Building the full chat transcript as a PDF now (this document).

Appendix - Code Changes by Task

Task 1 - Collision detection fix (frogger/game/collisions.py)

Bug: `check_collision` only checked if the vehicle's left-edge grid column equalled the frog's column. A vehicle many pixels wide could visually overlap the frog for ~23 frames before the left edge crossed into the frog's column.

Fix: Use `pygame.Rect.colliderect` on the same rectangles that are drawn to the screen.

```
def check_collision(frog, vehicles):
    frog_rect = frog.get_rect(CELL_SIZE)
    for v in vehicles:
        if v.row != frog.row:
            continue
        if frog_rect.colliderect(v.get_rect(CELL_SIZE)):
            return True
    return False
```

Task 2 - 3-life system (frogger/game/game_engine.py)

Added `self.lives = 3`, `self.state (playing / game_over)`, `_lose_life()` that decrements lives and either respawns the frog or sets game over, HUD Lives counter, Game Over banner, and freeze until R is pressed.

Task 3 - Goal detection and score (frogger/game/game_engine.py)

Reaching the goal row sets `STATE_WON` and adds a 100-point bonus. Each new row the frog reaches during an attempt adds 10 points. A `best_row` tracker prevents farming score by hopping back and forth. Score persists across lives.

Task 4 - 30-second countdown timer (frogger/game/game_engine.py, frogger/main.py)

`self.time_left` is decremented by real frame time (`dt` from `clock.tick(60) / 1000`, capped at 0.1 s). Timeout triggers `_lose_life("Time's up!")` which respawns the frog and resets the timer to 30 s. Timer turns red in the HUD when ≤ 5 s remain. Timer freezes on win or game over.

Unit tests (frogger/tests/test_game.py)

9 tests covering: collision overlap anywhere along vehicle width; no collision when clear; 3 hits → 2 → 1 → 0 lives / game over; frog frozen after game over, R restarts; reaching goal wins and scores; no score farming; timeout costs a life and resets timer; timer stops on game over and win; collision respawn resets timer.